

Manual do Repositório

Pasta a pasta, arquivo a arquivo. O que é cada parte do esqueleto e por que ela existe — um retrato descritivo do repositório como ele está hoje.

Fonte: **leitura direta do repositório construído** (70 arquivos, lidos um a um) · Status: retrato do estado atual do repo

1 O que é este repositório

É um **molde**, não um projeto. Ninguém trabalha "dentro dele" — cada projeto real do núcleo de dados nasce como uma **cópia independente** dele, roda um script de inicialização uma única vez, e segue vida própria. Não há repo-mãe, não há sincronização de volta.

A ideia que sustenta tudo é o **mini-cérebro nativo**: toda a inteligência de execução — regras, guias, procedimentos, decisões — viaja *dentro* do repo, como arquivo versionado. Quem clona consegue operar o padrão sozinho, sem depender do que está na cabeça de alguém.

2 A ideia organizadora: três territórios

Se você entender só uma coisa deste repo, entenda esta. Tudo está separado em três territórios, pela **natureza da informação**:

TERRITÓRIO	O QUE É	MUDA DE PROJETO PRA PROJETO?
<code>docs/</code>	Manual do molde. O que é o esqueleto, suas regras e guias	Não É igual em toda cópia
<code>.claude/</code>	Contexto deste projeto. Negócio, dados, stakeholders, glossário do domínio	Sim Cada cópia preenche
<code>pipeline/</code>	Onde o analista trabalha. O código de verdade	Sim É o trabalho

Regra de ouro: se a informação descreve o padrão, mora em `docs/` ; se descreve este projeto, mora em `.claude/` .

Fora desses três, sobra só infraestrutura: `platform/` (deploy), `tools/` (scaffolding e verificação), `tests/` , `.github/` e os cinco arquivos da raiz.

3 Mapa geral

```
raiz/          CLAUDE.md · README.md · pyproject.toml · uv.lock · .gitignore
.github/       workflows/ci.yml
docs/          GUIA-DO-ANALISTA.md · context/ · rules/ · guides/ · skills/
.claude/       context/ · rules/ · guides/ · skills/      + pasta oculta
pipeline/      common/ · config/ · data/ · entrypoints/ · models/ · notebooks/
platform/      databricks.yml · MLProject · conda.yaml · resources/
tools/         init.py · gen_conda.py · check.py · check_*.py
tests/         confest.py · test_*.py
```

4 A raiz — cinco arquivos, cada um com um motivo técnico

A raiz é uma **vitrine de ponteiros**, não um depósito. A regra: só fica na raiz o que *tecnicamente não funciona em outro lugar*. Gosto não é justificativa. Um script de CI (`check_root.py`) barra qualquer arquivo novo que apareça solto ali.

ARQUIVO	O QUE É	POR QUE ESTÁ NA RAIZ
<code>CLAUDE.md</code>	Ponto de entrada único , para IA ou pessoa. Explica a divisão molde x projeto e manda a IA ler as coisas na ordem certa antes de agir	É o caminho onde as ferramentas procuram
<code>README.md</code>	Porta de entrada humana — o que o GitHub renderiza. É template: tem trechos <code><!-- PREENCHER --></code>	É o que o GitHub renderiza na página do repo
<code>pyproject.toml</code>	Fonte única de dependências , mais a config do ruff, do pytest e do empacotamento	O <code>uv</code> só procura na raiz
<code>uv.lock</code>	As versões exatas de tudo. É o que garante que <code>uv sync</code> reconstrói sempre o mesmo ambiente	O <code>uv</code> só procura na raiz
<code>.gitignore</code>	O que não é versionado: <code>.venv</code> , caches, <code>mlruns/</code> , <code>mlflow.db</code> e os dados de <code>pipeline/data/</code>	O Git só lê neste caminho exato

Sobre o `CLAUDE.md` : apesar do nome, o conteúdo é neutro — serve para qualquer IA ou pessoa. Ele instrui a IA a primeiro *descobrir a fase do repo* (cópia nova ainda não inicializada x projeto já rodando) e só então agir. É o único ponto de entrada; não existe `AGENTS.md` .

Sobre o `pyproject.toml` : as dependências base são `numpy` , `pandas` , `pyarrow` , `scikit-learn` , `mlflow` , `pydantic` e `pyyaml` . Os extras `databricks` e `spark` são **opcionais** e nunca entram por padrão — o núcleo instala sozinho.

5 .github/ — a automação

Um arquivo só: `workflows/ci.yml` .

Roda a cada Pull Request e a cada push na `main` . Executa, em ordem: `ruff check` , `ruff format --check` , `pytest` e as **4 cancelas** (raiz mínima, manifesto em sync, MLflow centralizado, plataforma isolada). Se qualquer uma falha, o merge é barrado.

É este arquivo que impede as regras de virarem texto decorativo. Cada cancela é um *step nomeado*, para a violação aparecer identificável na interface do GitHub. E o CI não tem lógica própria escondida no YAML: ele chama os **mesmos scripts** de `tools/` que você roda na sua máquina.

6 docs/ — o manual do molde

Tudo aqui descreve **o esqueleto**, nunca o projeto. É idêntico em toda cópia.

`docs/GUIA-DO-ANALISTA.md`

O onboarding narrado, da cópia até o primeiro pipeline rodando. É o primeiro documento que uma pessoa nova lê. Não pressupõe nada.

docs/context/ — o que é o esqueleto

ARQUIVO	CONTEÚDO
<code>projeto.md</code>	O que é o esqueleto e para que serve
<code>arquitetura.md</code>	Estrutura-alvo, invariante central, princípio da raiz mínima (com a tabela de "por que cada arquivo da raiz não pode sair de lá"), anatomia de 5 arquivos, interface de dados neutra
<code>decisoes.md</code>	As 10 decisões ratificadas do molde, num arquivo só (não há pasta de ADRs)
<code>glossario.md</code>	Termos do molde: esqueleto, cópia, toggle, núcleo...

docs/rules/ — o que é inegociável

Oito arquivos numerados. Cada regra é curta, no formato **"faça / não faça"**, e cada uma tem um verificador automático ou é coberta pelos testes.

REGRA	O QUE EXIGE
<code>00-anatomia-de-5-arquivos</code>	Todo modelo tem exatamente 5 arquivos, com esses nomes. Nem 4, nem 6
<code>01-mlflow-centralizado</code>	Só <code>pipeline/common/tracking.py</code> importa mlflow. Mais ninguém
<code>02-fonte-unica-de-dependencias</code>	Dependência só entra pelo <code>pyproject.toml</code> . Nunca editar <code>conda.yml</code> à mão
<code>03-manifesto-sempre-em-sync</code>	Mudou o pyproject, regeira os artefatos derivados no mesmo commit
<code>04-raiz-minima</code>	Arquivo novo na raiz só com requisito técnico
<code>05-disciplina-de-nomes</code>	<code>snake_case</code> ; o nome do config bate com o nome do modelo
<code>06-plataforma-opcional</code>	O núcleo tem que rodar com <code>platform/</code> inexistente
<code>07-pipeline-config-driven</code>	Nada hardcoded: caminho, tabela, hiperparâmetro — tudo vem do YAML

docs/guides/ — como eu mexo em cada camada

Um guia por pasta do pipeline: `common.md` , `config.md` , `data.md` , `entrypoints.md` , `models.md` , `platform.md` . Todos com a mesma estrutura de três seções: **propósito**, **pode morar aqui**, **não pode morar aqui**. Servem para responder "onde eu ponho essa coisa nova?".

docs/skills/ — procedimentos passo a passo

Quatro receitas prontas: `adicionar-dependencia.md` , `adicionar-nova-fonte-de-dados.md` , `adicionar-novo-modelo.md` e `iniciar-projeto-novo-a-partir-do-esqueleto.md` . Cada uma é um passo a passo executável, apontando para um arquivo real do repo como referência viva.

7 .claude/ — o contexto deste projeto

Viaja **vazio de conteúdo real**: são templates com `<!-- PREENCHER -->`. É o que o analista preenche para tornar a cópia *dele*. Enquanto está em branco, a IA sabe operar o padrão, mas não sabe nada do domínio — e vai ter que adivinhar.

⚠ Pasta oculta

Use `ls -a` para enxergá-la.

.claude/context/

ARQUIVO	O QUE PREENCHER
projeto.md	Problema de negócio, fontes de dado, modelos do projeto, stakeholders. É o mais importante — é ele que ensina a IA sobre o domínio
arquitetura.md	Particularidades deste projeto que fogem do padrão (integrações, agendamentos, dependências entre pipelines)
decisoes.md	Decisões que valem só aqui (ex.: "métrica principal é F1 porque a classe positiva é rara")
glossario.md	Termos do negócio que aparecem no código e nos dados — o que é "churn", o que é "cliente ativo"

Cada um desses arquivos abre com um aviso apontando para o equivalente em `docs/context/`, para não confundir os dois mundos.

.claude/rules/ , .claude/guides/ , .claude/skills/

Cada uma tem só um `README.md` explicando que é espaço para **acréscimos específicos deste projeto**, e que as versões do molde estão em `docs/`. Não duplicar as do molde aqui.

★ 8 pipeline/ — onde o analista trabalha PRINCIPAL

Todo o código que o analista cria e edita. É a única pasta de código na raiz.

Detalhe técnico: usa **src-layout**. A pasta `pipeline/` entra no `sys.path`, então os imports no código são de primeiro nível — escreve-se `from common.config import ...`, não `from pipeline.common.config import ...`.

pipeline/common/ — lógica compartilhada entre modelos

ARQUIVO	O QUE FAZ
<code>__init__.py</code>	Marca o pacote raiz do núcleo compartilhado
<code>config.py</code>	Os schemas pydantic (<code>DataSourceConfig</code> , <code>TrackingConfig</code> , <code>BaseModelConfig</code>) e o <code>load_config()</code> que lê o YAML e valida. É aqui que um config errado é pego cedo, com mensagem clara
<code>data_source.py</code>	O contrato. Define o <code>Protocol DataSource</code> (dois métodos: <code>read</code> e <code>write</code>) e a fábrica <code>build_data_source()</code> que escolhe a implementação conforme o config. Este arquivo <i>não</i> importa spark nem sqlalchemy — é contrato puro
<code>tracking.py</code>	O único arquivo do repo que fala com MLflow. A classe <code>Tracker</code> envolve o mlflow e encapsula a diferença entre rodar dentro e fora do Databricks
<code>splits.py</code>	Divisões treino/teste: <code>random_split()</code> (com estratificação opcional) e <code>temporal_split()</code> (por data de corte ou por fração final, sem embaralhar — evita vazamento do futuro)

pipeline/config/ — a configuração por modelo

`exemplo_modelo.yaml` é o template, todo comentado. Contém quatro blocos: `data_source` (tipo e parâmetros da fonte), `tracking` (experimento e backend do MLflow), `output_dir` e `params` (seleção de colunas, estratégia de split e hiperparâmetros do estimador). O nome do arquivo bate com o nome da pasta do modelo. Copie e renomeie para o seu modelo real.

pipeline/data/ — de onde o dado vem

`sources/` traz as **implementações concretas** do contrato. Cada uma carrega suas dependências pesadas *lazy* (só quando o método é chamado), para que o repo importe mesmo sem pyspark ou sqlalchemy instalados.

ARQUIVO	FONTE
<code>parquet_source.py</code>	Arquivo/diretório Parquet local. É o default e funciona sempre (pandas + pyarrow são base)
<code>spark_table_source.py</code>	Tabela Spark (ex.: Unity Catalog no Databricks). Precisa do extra <code>spark / databricks</code>
<code>sql_source.py</code>	Banco relacional via SQLAlchemy, por query ou por tabela

Também é aqui que entra o dado bruto, em `raw/` — que é gitignored e por isso não aparece no repo versionado.

pipeline/entrypoints/ — os launchers finos

Casca de entrada, **zero lógica de negócio**: parseiam `--config` e chamam o orchestrator.

- `run_local.py` — execução local
- `run_serverless.py` — execução na plataforma (removido pelo init se Databricks = Não)

Os dois chamam **exatamente o mesmo núcleo**. A diferença de plataforma mora no config, nunca no código.

pipeline/models/ — a anatomia de 5 arquivos

Cada modelo é uma pasta em snake_case com exatamente estes cinco arquivos. O exemplo_modelo/ é a referência viva:

ETAPA	ARQUIVO	PAPEL	RESTRIÇÃO
1/5	prepare_data.py	Pede o dado à interface, seleciona features/alvo, divide treino/teste. Devolve um PreparedData	Não sabe de onde o dado vem
2/5	build_model.py	Constrói o estimador a partir dos hiperparâmetros do config	Não treina, não faz I/O
3/5	train.py	Recebe modelo + dados e devolve o modelo treinado	Não lê config, não faz I/O
4/5	evaluate_model.py	Calcula accuracy, f1, precision e recall no conjunto de teste	Devolve as métricas; não as loga
5/5	orchestrator.py	A espinha dorsal. Amarra config → prepare → build → train → evaluate e loga tudo	É o único que fala com tracking

Os quatro primeiros não importam mlflow — nem indiretamente. Quem loga é sempre o orchestrator, e sempre via common/tracking.py .

pipeline/notebooks/

Onde ficam os .ipynb : análise exploratória, tratamento de base, merges. A regra que o README da pasta deixa clara: **notebook é rascunho, não produção**. Quando uma lógica de preparação amadurece, ela migra para prepare_data.py (ou para common/ , se for reutilizável entre modelos).

9 platform/ — a casca opcional de Databricks

Fica na **raiz**, fora de `pipeline/`, porque é infra de deploy e não código de análise. Só sobrevive se o toggle for Sim — se for Não, o `init.py` apaga a pasta inteira e nada quebra.

ARQUIVO	O QUE É
<code>databricks.yml</code>	O Asset Bundle: nome, variáveis (catálogo, schema, caminho do experimento) e os ambientes <code>dev</code> / <code>prod</code> . Vem com placeholders <code><PREENCHER: ...></code> que a cópia completa
<code>resources/treino_exemplo_modelo.yml</code>	O job de treino serverless. Aponta para o entrypoint , nunca direto para o núcleo, e passa só o caminho do config
<code>MLProject</code>	Manifesto MLflow, para rodar o pipeline como um MLflow Project
<code>conda.yml</code>	⚠ Arquivo gerado — não editar. Produzido por <code>tools/gen_conda.py</code> a partir do pyproject

10 tools/ — scaffolding e verificação

Todos usam **só a biblioteca padrão**, porque precisam rodar antes mesmo do `uv sync` existir.

ARQUIVO	O QUE FAZ
<code>init.py</code>	O scaffolder. Roda uma vez após a cópia: renomeia o modelo de exemplo e seu config, substitui os placeholders de nome do projeto, aplica o toggle Databricks (podando <code>platform/</code> , o <code>run_serverless.py</code> e o extra do pyproject se for Não) e se autodestrói ao terminar
<code>gen_conda.py</code>	Gera o <code>platform/conda.yml</code> a partir do pyproject. Determinístico: mesma entrada, mesma saída, byte a byte
<code>check.py</code>	O portão local. Roda todas as verificações de uma vez, na mesma ordem do CI. Diferente do CI, não para no primeiro erro — mostra todas as falhas juntas. Aceita <code>--fix</code>
<code>check_root.py</code>	Cancela 1 — raiz mínima. Mantém a allowlist explícita e, se achar um intruso, sugere para onde ele deveria ir
<code>check_manifest.py</code>	Cancela 2 — manifesto em sync. Regera o conda.yml e falha se diferir do versionado. Se <code>platform/</code> não existe, passa informando que pulou
<code>check_mlflow.py</code>	Cancela 3 — MLflow centralizado. Usa AST, não grep: uma docstring que <i>menção</i> mlflow não é violação, e <code>import mlflow as mf</code> é
<code>check_platform.py</code>	Cancela 4 — plataforma isolada. Varre <code>pipeline/</code> procurando qualquer import de <code>platform</code>

Por que o init.py se autodestrói

Ele só faz sentido uma vez. Depois disso é código morto num repo que preza por raiz mínima. E a ausência do arquivo *já é* o estado "já executou" — autoevidente, impossível de dessincronizar, e garante por física (não por disciplina) que ninguém roda duas vezes. Ele não se perde: continua no histórico do git do esqueleto.

11 tests/ — a suíte do molde

Testa o **encanamento**, não a qualidade do modelo. A pergunta que respondem é: "*o esqueleto ainda funciona e ainda segue as próprias regras?*"

ARQUIVO	O QUE PROVA
<code>conftest.py</code>	Fixtures compartilhadas: dado sintético pequeno + config apontando para ele
<code>test_core.py</code>	O núcleo neutro: config, splits, fábrica de fonte. Inclui a prova de que importar a fonte Spark não exige pyspark instalado
<code>test_exemplo_modelo.py</code>	O pipeline de referência ponta a ponta, mais os contratos estruturais (anatomia de 5 arquivos, isolamento do MLflow)
<code>test_gen_conda.py</code>	Que o conda.yml é derivado, determinístico, e que não dá pra commitar fora de sync sem o CI ver
<code>test_init.py</code>	O scaffolder rodando de verdade , em modo não-interativo, numa cópia temporária — nos dois toggles
<code>test_invariante_plataforma.py</code>	O invariante central: o núcleo roda sem a pasta <code>platform/</code>
<code>test_smoke.py</code>	Checagem mínima: o pacote e as dependências base importam

Cancelas ≠ testes

As cancelas (`tools/check_*.py`) verificam *regras de arquitetura*; os testes (pytest) verificam se o *código funciona*. O `check.py` e o CI rodam as duas coisas juntas.

12 Os cinco princípios que amarram tudo

- 1 **Invariante do toggle** — o núcleo é idêntico com ou sem Databricks. `platform/` é a casca inteira que liga/desliga; nada de `pipeline/` importa de lá. Por isso apagar a pasta é operação limpa.
- 2 **Raiz mínima** — só fica na raiz o que tem requisito técnico. Gosto não é justificativa.
- 3 **Pipeline config-driven** — nenhum caminho, tabela, hiperparâmetro ou nome hardcoded. Tudo vem do YAML.
- 4 **MLflow centralizado** — um arquivo só fala com o mlflow. Isso confina num único ponto a diferença entre rodar dentro e fora do Databricks.
- 5 **Cancelas de CI** — cada regra estrutural tem um script que a verifica. Regra que não é verificada é regra ignorada.

13 Como um analista usa, na prática

- 1 **Obtém a própria cópia** do esqueleto (repo novo, sem o `.git` do molde). *⚠ Nunca rodar o init dentro do repo-modelo — ele apaga pastas.*
- 2 `python3 tools/init.py` — script de terminal, não é IA. Pergunta o nome do projeto, Databricks (S/N) e o nome do primeiro modelo. Poda o que não se aplica e se autodestrói.
- 3 `uv sync` — só agora, nunca antes: o init pode ter mexido no pyproject.
- 4 **Preenche** `.claude/context/projeto.md` — é o que torna a cópia dele e ensina a IA sobre o domínio.
- 5 **Trabalha:** config em `pipeline/config/`, modelo em `pipeline/models/<nome>/`, roda com `uv run python pipeline/entrypoints/run_local.py --config pipeline/config/<modelo>.yaml`.
- 6 **Antes de commitar:** `python3 tools/check.py`.